



Experiment 09

UDP Socket Programming in Python

Name: Deven

UID: 24BCF10060

Aim

To implement a simple client-server communication model using UDP (User Datagram Protocol) sockets in Python, and to understand the connectionless nature of UDP communication.

Learning Objectives

1. Understand the connectionless, datagram-based nature of UDP.
2. Create a UDP socket using `SOCK_DGRAM`.
3. Bind a UDP server socket to an address without calling `listen()/accept()`.
4. Send and receive individual datagrams using `sendto()/recvfrom()`.
5. Compare UDP's connectionless behaviour with the connection-oriented TCP model.
6. Observe and verify two-way UDP communication in real time.

Theory

UDP: Connectionless Communication

UDP (User Datagram Protocol) is a *connectionless*, lightweight transport protocol. Unlike TCP, it does not perform a handshake and does not guarantee delivery, ordering, or duplicate-protection of data – each message (*datagram*) is sent independently, addressed to a specific destination IP and port, with no persistent connection state kept by either side. This removes handshake and connection-management overhead, making UDP faster and lighter for applications that can tolerate occasional loss.

Key Socket Calls Used

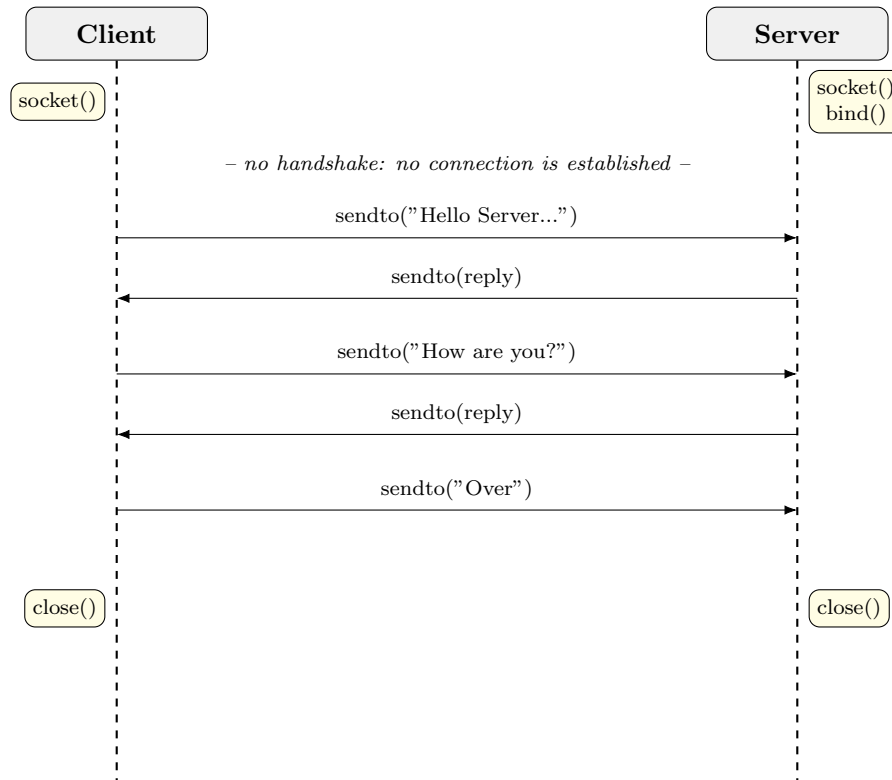
- `socket.socket(AF_INET, SOCK_DGRAM)` – creates an IPv4 UDP socket (`SOCK_DGRAM` means a connectionless datagram socket, as opposed to TCP's `SOCK_STREAM`).
- `bind((host, port))` – still needed on the server so the OS knows which port to deliver incoming datagrams to; there is no equivalent to TCP's `listen()/accept()`.
- `sendto(data, address)` – sends one self-contained datagram to a specific (IP, port), with no prior connection needed.
- `recvfrom(bufsize)` – waits for the next incoming datagram and returns both its data *and* the sender's address – this is how the server learns where to send its reply, since there is no persistent connection to remember the client.
- `close()` – releases the socket.

Real-World Use Cases

UDP is preferred wherever low latency matters more than guaranteed delivery: DNS lookups, live video/audio streaming, VoIP calls, and online multiplayer games, where a late or slightly lossy packet is less harmful than the delay a retransmission-and-acknowledgement scheme would add.

Communication Sequence

Unlike TCP, there is no handshake step – the client can call `sendto()` immediately after creating its socket. The diagram below shows the same two-message exchange as the TCP experiment, but purely as independent datagrams.





Procedure

Part A – server.py

```
1 import socket
2
3 ## Creating a UDP Socket
4 server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
5 ## A socket is a software endpoint through which a program sends/
   receives data.
6
7 server_socket.bind(("127.0.0.1", 5000))
8
9 print("UDP Server started")
10 print("Waiting for messages...")
11
12 while True:
13     message, client_address = server_socket.recvfrom(1024)
14     ## recvfrom() returns both the data AND the sender's address,
15     ## since there is no persistent connection to remember it.
16     message = message.decode()
17     print("Client:", message)
18
19     if message == "Over":
20         break
21
22     reply = input("Enter reply: ")
23     server_socket.sendto(reply.encode(), client_address)
24
25 server_socket.close()
26 print("Server closed")
```



Part B – client.py

```
1 import socket
2
3 client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4 server_address = ("127.0.0.1", 5000)
5
6 print("UDP Client started")
7
8 while True:
9     message = input("Enter message: ")
10    client_socket.sendto(message.encode(), server_address)
11    ## Every sendto() is independently addressed -- no connect()
        needed.
12
13    if message == "Over":
14        break
15
16    reply, server_address_received = client_socket.recvfrom(1024)
17    print("Server:", reply.decode())
18
19 client_socket.close()
20 print("Client closed")
```

Part C – Running Order

The server is started first so it is ready to `recvfrom()` as soon as a datagram arrives. The client is then started, and the two exchange datagrams interactively until the client sends "Over". Unlike the TCP experiment, the server does not block waiting for a connection to be *opened* – it simply waits for the next datagram to *arrive*.

Verification / Output

Server terminal

```
UDP Server started
Waiting for messages...
Client: Hello Server, this is client
Enter reply: Hi Client, message received!
Client: How are you?
Enter reply: I am fine, thanks for asking!
Client: Over
Server closed
```

Client terminal

```
UDP Client started
Enter message: Hello Server, this is client
Server: Hi Client, message received!
Enter message: How are you?
Server: I am fine, thanks for asking!
Enter message: Over
Client closed
```

Result

Two-way communication was successfully implemented between a UDP client and server on 127.0.0.1 port 5000 using `sendto()/recvfrom()`, with no connection handshake at any point. Each message was sent and received as an independent datagram, and the server correctly used the sender's address returned by `recvfrom()` to route its reply back to the client – confirming the connectionless behaviour of UDP.

Discussion – TCP vs UDP

Aspect	TCP (Experiment 08)	UDP (this experiment)
Connection	Connection-oriented (handshake)	Connectionless (no handshake)
Socket type	SOCK_STREAM	SOCK_DGRAM
Setup calls	bind, listen, accept, connect	bind only
Reliability	Guaranteed delivery & order	No delivery/order guarantee
Overhead	Higher (handshake, state)	Lower (stateless datagrams)
Addressing	Fixed once connected	Given with every send/recv
Typical use	Web, email, file transfer	DNS, streaming, gaming, VoIP

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood UDP's connectionless, datagram-oriented communication model.
- Implemented a UDP server using `bind()` and `recvfrom()`, without `listen()/accept()`.
- Implemented a UDP client that sends independent datagrams using `sendto()`, without `connect()`.
- Observed how the server uses the address returned by `recvfrom()` to reply to the correct client.
- Compared UDP and TCP directly, and identified when each protocol is the appropriate choice.